First Class Agency

Agent Portal & CRM

Complete Build Guide

Everything required to build this website from a blank repository to a live, fully-functional agent portal — hosting, design system, database, authentication, billing, telephony, lead automation, and deployment. Written so it can also be used as a clone recipe for a new brand.

Prepared for Malikai Lee
Live domain: agents.firstclassagency.info
Stack: Static HTML/CSS/JS · GitHub Pages · Supabase · Stripe · Twilio · GroupMe
Document generated May 19, 2026

Contents

0 Overview & Architecture

The portal is a private, invite-only web app for insurance agents. It bundles training tracking, a custom CRM, a power-dialer paywall, SMS messaging, prepaid billing, a live sales leaderboard, and automated lead ingestion — all behind a single login.

Architecture at a glance

The frontend is plain static files (no framework, no build step) served by GitHub Pages. All dynamic behavior happens through Supabase: Postgres holds the data, Row-Level Security (RLS) enforces who can see what, and Edge Functions (Deno) handle anything that needs a server — Stripe charges, inbound webhooks, lead ingestion. Stripe powers payments, Twilio (planned) powers calling and texting, and a GroupMe bot feeds the leaderboard.

Layer	Technology	Responsibility
Hosting	GitHub Pages + CNAME	Serves static files at the custom domain
Frontend	HTML / CSS / vanilla JS	All UI; one stylesheet, 8 shared JS modules
Database	Supabase Postgres	Profiles, leads, billing, leaderboard, dispositions
Security	Supabase RLS policies	Per-agent + admin-hierarchy data scoping
Server logic	Supabase Edge Functions (Deno)	Stripe, GroupMe, lead-ingest webhooks
Payments	Stripe Payment Element	Inline card top-ups + webhooks
Telephony	Twilio (planned)	Voice power-dialer + A2P SMS
Leaderboard	GroupMe webhook	Parses '$NNNN CARRIER' chat messages
Lead intake	Edge fn + Apps Script	Google Sheets / vendor rows into the CRM

Repository inventory

- **index.html** + **reset-password.html** — entry gate and password reset.
- **agent/** — 16 portal pages (dashboard, training, dialer[CRM], power-dialer, messages, team, settings, account, leaderboard, portals, quick-links, resources, promotion, carriers, builders, admin).
- **assets/css/styles.css** — the entire design system in one file.
- **assets/js/** — 8 modules: agent-auth, theme, main, ranks, training-schedule, credits-pause, money-explosion, gate-particles.
- **assets/img/** — logo, favicon, PWA icons, content imagery.
- **supabase/** — onboarding-setup.sql + 26 numbered migrations + 4 edge functions.
- **manifest.webmanifest** + **CNAME** + **.nojekyll** — PWA, domain, Pages config.

1 Prerequisites & Accounts

Create these accounts before writing any code. All have free tiers adequate to launch; costs scale with usage.

Account	Used for	Cost to start
GitHub	Code hosting + GitHub Pages deploy	Free
Domain registrar	Custom domain (e.g. agents.yourbrand.com)	~$12/yr
Supabase	Database, auth, edge functions	Free tier, then ~$25/mo
Stripe	Card payments + webhooks	Free; 2.9% + 30c per charge
Twilio	Voice + SMS (when live)	Pay-as-you-go
GroupMe	Leaderboard message feed	Free
Google	Apps Script lead auto-sync	Free

Local tooling

- **Git** — version control and pushing to GitHub.
- **Node.js** — to run the Supabase CLI.
- **Supabase CLI** — install with `npm install -g supabase`. Used to deploy edge functions.
- A code editor (VS Code or similar).

2 Repository & Hosting Setup

1. Create a new GitHub repository (public is fine; the gate is a soft lock, not real security — sensitive data lives behind Supabase auth, not in the static files).
2. Add an empty **.nojekyll** file at the repo root so GitHub Pages serves files literally and doesn't try to process them as a Jekyll site.
3. Add a **CNAME** file containing only your bare domain, e.g. `agents.yourbrand.com`.
4. Push the project to the **main** branch.
5. In the repo: Settings → Pages → Source = **Deploy from a branch**, branch **main**, folder **/ (root)**.
6. Configure DNS at your registrar: a CNAME record pointing your subdomain at `<username>.github.io`. GitHub's Pages settings shows the exact target and a 'Verify domain' step.
7. Enable **Enforce HTTPS** once the certificate provisions (a few minutes).

> *Every change goes live the same way for the life of the project: commit, then* `git push`*. GitHub Pages redeploys within ~60 seconds. There is no build step.*

3 Brand System (the reskin layer)

All brand identity is concentrated so the whole site can be re-skinned by editing a handful of values. This is the layer you change when cloning for a new brand.

Color tokens

The single source of truth is the `:root` block at the top of **assets/css/styles.css**. First Class uses:

```
--accent:        #0078ab;   /* brand blue */
--bg-1:          #1e1e1e;   /* dark surface */
--text:          #ffffff;   /* on dark */
--accent-bright: #1aa3df;   /* hover / highlights */
```

There is a full dark theme and a light theme (under `[data-theme='light']`). Change the variables once and every page, button, gradient, and chart re-colors automatically. Never hardcode hex values in individual pages.

Typography

Inter for UI text, JetBrains Mono for numbers and code-like values (phone numbers, tokens). Both load from Google Fonts in each page head.

Assets to replace

- **assets/img/logo.png** — header logo.
- **assets/img/favicon.svg** — browser tab icon.
- **assets/img/apple-touch-icon.png**, **icon-192.png**, **icon-512.png**, **icon-512-maskable.png** — PWA / home-screen icons.
- Content imagery: **top-writer.png**, **team-call.jpg**, **agency-record.jpg**.

Manifest & metadata

- **manifest.webmanifest** — name, short_name, theme_color, background_color.
- Header brand text ('First Class · Agent') across page headers.
- **<title>** tags and the README.

4 Frontend Foundation

Build these before any page, since every page depends on them.

4.1 — The stylesheet

assets/css/styles.css holds the complete design system: CSS variables, the gate, headers, sidebar nav, cards, buttons, the CRM table, the Power Dialer UI, Messages two-pane layout, billing/settings panels, the leaderboard, and the lead auto-sync admin table. One file, theme-aware via CSS variables.

4.2 — Shared JavaScript modules

File	Responsibility
agent-auth.js	Supabase client init, session gate, role resolution, exposes window.fcAuth + window.fcaSupabase + window.fcaSup
theme.js	Loads / persists / applies dark|light theme before paint
main.js	Header behavior, scroll reveals, magnetic CTAs, mobile nav
ranks.js	Monthly rank tiers (Copper/Silver/Gold...) + badge rendering
training-schedule.js	window.FCA_SCHEDULE — the Mon-Fri call lineup (data only)
credits-pause.js	Master Switch: pauses credit-consuming actions; auto-pause at $0
money-explosion.js	Brand-blue / green bill animation on dialer entry + Sold
gate-particles.js	Animated particle background on the login gate

4.3 — The entry gate

index.html is the entry point. Visitors authenticate via Supabase; a valid session routes into **agent/dashboard.html**. **reset-password.html** handles the forgot-password flow. Sign-out clears the session and returns to the gate.

5 Supabase Backend & Database Schema

1. Create a new Supabase project. Note the **Project URL** and the **publishable (anon) key**.
2. In **assets/js/agent-auth.js**, set SUPABASE_URL and SUPABASE_KEY to your project's values. (These are safe to expose client-side; RLS is what protects the data.)
3. Open Supabase → SQL Editor and run the migrations **in numeric order**, starting with onboarding-setup.sql, then 02 through 26. Each is idempotent — safe to re-run.

Migration sequence

Run these in order. Each builds on the previous.

File	What it creates
onboarding-setup.sql	profiles mirror, user_roles, onboarding_progress, RLS, signup trigger
02-hierarchy.sql	manager_id, super_admin role, scope-based RLS (two-level)
03-weeks-redesign.sql	8-week onboarding grid model
04-task-progress.sql	Per-task progress tracking
05-admin-calendly.sql	Admin Calendly URLs + profile visibility
06-training-attendance.sql	Training-call attendance tracking
07-leaderboard.sql	leaderboard_entries + get_leaderboard() RPC
08-rank-helpers.sql	Monthly rank tier helpers
09-agent-stats.sql	Agent personal stats
10-top-producer.sql	Top-producer flag (gold ring / glow)
11-leaderboard-top-producer.sql	Adds top-producer flag to month premiums
12-crm-leads.sql	leads + call_attempts tables, RLS, daily-stats RPC
13-dialer-fields.sql	Dialer v2 lead field additions
14-dialer-settings.sql	dialer_settings JSON on profiles
15-billing.sql	Prepaid credit balance, Stripe customer fields, ledger
16-sms-messages.sql	SMS conversations + messages
17-lead-distribution.sql	Round-robin / single-agent distribute RPC
18-lead-type.sql	lead_type column + import support
19-dispositions.sql	13-disposition list + color coding
20-drip-defaults.sql	Drip exclusion defaults
21-power-dialer.sql	Power Dialer subscription state
22-credits-pause.sql	credits_paused flag + is_credits_paused() RPC
23-auto-pause-on-empty.sql	Trigger: auto-pause when balance hits $0

File	What it creates
24-lead-webhook-tokens.sql	agent_lead_token + generator + rotate RPC
25-admin-rotate-lead-token.sql	Admin can rotate a downline agent's token
26-fix-signup-trigger.sql	Hardens the new-user signup trigger

The RLS model

Three visibility rules run through the whole schema: an agent sees only their own rows; a team admin sees their direct reports; a super_admin sees everyone. Most tables carry an `agent_id` and policies keyed to `auth.uid()`, `is_admin()`, and `is_super_admin()`.

6 Authentication, Roles & Hierarchy

Supabase Auth handles email/password sign-in. A database trigger mirrors each new `auth.users` row into **public.profiles** so admins can list agents by name. Signup is invite-only — you provision accounts rather than allowing open registration.

Roles

- **agent** — default; sees only their own data.
- **admin** — team lead; sees direct reports (manager_id points to them).
- **super_admin** — sees and manages everyone.

Admin-only UI is revealed client-side via `[data-admin-only]` elements that agent-auth.js un-hides once it resolves the role. RLS is the real enforcement; the hiding is just UX.

> *Dialer/CRM access during build was gated to an allowlist of emails in agent-auth.js (DIALER_ALLOWLIST). To roll the CRM out to everyone, remove that allowlist block and the [data-dialer-only] hiding.*

7 Building the Pages

Each page is a standalone HTML file sharing the same header, sidebar, CSS, and JS. Build order roughly follows dependency — dashboard and training first, CRM and its tabs later.

Page	Purpose
dashboard.html	KPIs, today's training card, top-writer + recap showcase
training.html	8-week onboarding grid, progress checklists, video lessons
dialer.html	The CRM — leads list, filters (incl. lead type), click-to-call, CSV import with column mapping, pagination, CRM tab
power-dialer.html	$110/mo paywall + mocked single-line power-dialer UI
messages.html	Two-pane SMS inbox with credits-pause gate
team.html	Admin: lead distribution + bulk import + aged-lead reassignment
settings.html	General, Phone Setup, Auto-sync leads (admin), Billing tabs
account.html	Profile, avatar upload, theme, Calendly URL
leaderboard.html	Live monthly premium ranking from GroupMe feed
admin.html	Team training-progress + attendance breakdown
portals / quick-links / resources	Reference info

Cross-cutting mechanics

- **Master Switch** (credits-pause.js) injects a toggle into the user menu that halts credit-consuming actions; auto-engages at $0 balance via a DB trigger.
- **Money explosion** (money-explosion.js) fires on sidebar entry to the CRM (blue bills) and on the Sold disposition (green bills).
- **Dispositions** — 13 outcomes with color coding and SMS-trigger flags; drip exclusion logic.

8 Stripe Billing & Prepaid Credits

Agents pre-load credits that are consumed by SMS and voice usage. Top-ups happen inline (no redirect) via the Stripe Payment Element.

1. Create a Stripe account; grab the **publishable** and **secret** keys (use test keys until go-live).
2. Store the secret key + webhook signing secret as Supabase edge-function secrets.
3. Deploy **create-payment-intent** — returns a client secret the Payment Element uses to collect a card inline on the Billing tab.
4. Deploy **stripe-webhook** — handles payment_intent.succeeded (credit the balance + save card metadata), payment_failed (log), and charge.dispute.created (freeze account). Deploy it with `--no-verify-jwt` since Stripe sends no JWT.
5. In Stripe, add a webhook endpoint pointing at the deployed stripe-webhook URL; subscribe to the three events above.

> *Watch the test-vs-live trap: a test-mode webhook secret will reject live events with a 400 invalid-signature, and vice-versa. Keep the secret and the endpoint mode matched.*

The prepaid model

Balance lives in `credit_balance_cents` on profiles. The Master Switch and the $0 auto-pause trigger protect against overdraft. Rates (SMS, voice/min, number rental) are shown on the Billing tab's rate card.

9 Leaderboard (GroupMe feed)

The leaderboard ranks agents by premium written, sourced automatically from the team's GroupMe chat where agents post their sales as '$NNNN CARRIER' messages.

1. Create a GroupMe bot pointed at your sales group; set its callback URL to the deployed **groupme-webhook** edge function.
2. Each chat message hits the function, which parses the dollar amount + carrier and inserts a row into **leaderboard_entries** (deduped by GroupMe message id).
3. **leaderboard.html** calls the get_leaderboard() RPC to render day/week/month rankings, grouped by sender name.

> *Because grouping is by sender name, an agent who changes their GroupMe display name fragments into multiple entries. Fix by running an UPDATE on leaderboard_entries to unify the old names to the current one.*

10 Lead Webhook Ingest

Leads flow into the CRM automatically from Google Sheets or any vendor that can POST JSON — no manual CSV uploads required.

How it works

1. Migration 24 gives every agent a unique **agent_lead_token**. Their webhook URL is `.../functions/v1/leads-webhook-ingest/<token>`.
2. The **leads-webhook-ingest** edge function validates the token, maps incoming columns to lead fields (header-aware, with a value-sniff fallback and a forgiving DOB parser), de-dupes by phone, and inserts into the leads table.
3. For Google Sheets: an Apps Script `onChange` trigger reads row 1 as headers and POSTs each new row as `{headers, row}`. It gates on the phone column being filled so half-typed rows aren't sent.
4. Admins manage every agent's URL from Settings → Auto-sync leads (admin-only tab) — copy URL, copy pre-baked script, or rotate a token.

Accepted header names map case-insensitively: Phone/Cell/Mobile, First/Last/Full Name, Email, State, Address, DOB/Date of Birth, Lead Type, Notes, Source.

> *Deploy leads-webhook-ingest with --no-verify-jwt — external services (Apps Script, Facebook Lead Ads) don't send a Supabase JWT.*

11 Twilio Telephony (planned)

The Power Dialer UI and click-to-call are built and mocked; wiring live telephony is the remaining phase.

- **Phone numbers** — each agent rents a Twilio number (managed on Settings → Phone Setup): buy, rotate, release, and set a forward-to number.
- **Voice** — single-line power dialer with disposition grid; Twilio Conference API enables listen / whisper / barge for call coaching.
- **Answering Machine Detection** — set MachineDetection on the dial so voicemails don't count as contacts; carry a counts_as_contact flag on dispositions.
- **SMS** — requires A2P 10DLC brand + campaign registration before sending. Usage bills against prepaid credits.

12 Edge Function Deployment

The four edge functions live in **supabase/functions/**. Deploy them with the Supabase CLI.

```
# one-time
npm install -g supabase
supabase login
supabase link --project-ref <your-project-ref>

# deploy (webhooks must skip JWT verification)
supabase functions deploy create-payment-intent
supabase functions deploy stripe-webhook        --no-verify-jwt
supabase functions deploy groupme-webhook        --no-verify-jwt
supabase functions deploy leads-webhook-ingest   --no-verify-jwt
```

Set secrets (Stripe keys, webhook signing secret) via the dashboard or `supabase secrets set KEY=value`. SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY are injected automatically.

> *Re-deploy a function any time its code changes — the live function does NOT update from a git push. Editing the .ts file and pushing to GitHub does nothing until you re-run the deploy command.*

13 PWA / Installable App

The portal installs to a phone home screen as a standalone app.

- **manifest.webmanifest** — app name, icons, theme/background color, standalone display, portrait orientation.
- Apple meta tags in each page head (apple-mobile-web-app-capable, status-bar-style, title) for iOS install behavior.
- Icon set in assets/img: 192, 512, and 512-maskable, plus apple-touch-icon.

14 Brand-Swap Checklist (cloning)

To stand up this portal for a new brand, work top to bottom:

1. Duplicate the repository; set a new **CNAME** and configure DNS.
2. Edit the `:root` color tokens in styles.css to the new brand palette.
3. Replace all assets in assets/img (logo, favicon, PWA icons, content imagery).
4. Update manifest.webmanifest (name, colors) and all header brand text + titles.
5. Create a fresh Supabase project; put its URL + publishable key into agent-auth.js.
6. Run onboarding-setup.sql then migrations 02-26 in order in the new project.
7. Create a fresh Stripe account; set edge-function secrets; add the webhook endpoint.
8. Create a GroupMe bot for the new team; point it at groupme-webhook.
9. Deploy all four edge functions to the new project (webhooks with --no-verify-jwt).
10. Provision the first admin/super_admin account and invite agents.
11. Hand each agent their lead-webhook URL (or set up their Google Sheet Apps Script).

> *Never reuse another brand's Supabase, Stripe, or Twilio keys. Each clone is a completely separate backend; only the frontend code and SQL migrations are shared.*

15 Operations & Maintenance

Routine tasks once the portal is live:

Task	How
Update weekly training schedule	Edit window.FCA_SCHEDULE in assets/js/training-schedule.js; commit + push
Fix a fragmented leaderboard name	UPDATE leaderboard_entries SET sender_name=... WHERE sender_name IN (...)
Onboard a new agent's lead sheet	Settings -> Auto-sync leads -> copy that agent's script into their Sheet's Apps Script
Rotate a leaked lead token	Settings -> Auto-sync leads -> Rotate (admin)
Push code changes live	git push (Pages redeploys in ~60s)
Push edge-function changes live	supabase functions deploy <name> [--no-verify-jwt]
Take the site down temporarily	Swap index for a maintenance page + push, or disable Pages in repo settings

---

This document is a living reference. As the portal evolves — Twilio going live, new pages, schema changes — keep the migration list and the page table current so a future rebuild stays accurate.